Software Requirements Specification

LABCRAFT- Automated Practical Lab & Code Submission Grader

---

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of the Automated Practical Lab & Code Submission Grader (APLCSG). It is intended to guide system design, development, testing and deployment.

1.2 Scope

This project is basically a web-based platform where teachers create virtual classrooms and assign programming tasks, and students write, run and submit code from an in-browser editor. Submissions are executed asynchronously in an isolated sandbox against public and hidden test cases. The automated result is then forwarded to the teacher, who reviews it and explicitly approves the submission to award final marks. The system covers registration, teacher approval by an Admin, class-code based enrolment, assignment management, sandboxed grading, the teacher review pipeline and classroom analytics.

1.3 Targeted Audience

- Teachers and Lab Instructors: To easily create classrooms, post programming assignments, define hidden test cases, review student submissions with comments, and track class performance analytics.
- Students: To join virtual classrooms via Class Codes, solve programming assignments in an integrated Monaco code editor, receive instant automated test feedback, and view final grades.
- System Administrators: To manage user accounts, oversee platform health, and approve newly registered teacher accounts before they can create classrooms.
- Development Team: To design, implement, test, and maintain the platform using React.js, Django REST Framework, PostgreSQL, and Docker sandboxing.
- Project Evaluators & Guides: To assess the technical implementation, architectural design, security, and real-world applicability of the system.

1.4 Definitions, Acronyms and Abbreviations

Term	Definition
APLCSG	Automated Practical Lab & Code Submission Grader
SRS	Software Requirements Specification
RBAC	Role-Based Access Control
DRF	Django REST Framework

Term	Definition
JWT	JSON Web Token
Class Code	Unique alphanumeric code generated for each classroom, used by students to join
Public Test Case	Test case whose input and expected output are visible to the student
Hidden Test Case	Test case used for grading whose details are not shown to the student
Sandbox	Isolated runtime (Docker / subprocess) with limits on time, memory and system calls
Pending Approval	State of a newly registered Teacher awaiting Admin approval

1.5 References

- Django, Django REST Framework, SimpleJWT, Celery and drf-spectacular official documentation
- OpenAPI Specification 3.0
- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications

---

2. Overall Description

2.1 Product Perspective

APLCSG is a new, standalone web application with a React single-page frontend, a Django REST API backend, a Celery worker for asynchronous grading, Redis as message broker, and PostgreSQL as the primary datastore. Code execution is delegated to an isolated sandbox so that untrusted student code never runs in the API process.

2.2 User Classes and Characteristics

User Class	Description	Access Rule
Student	Joins classrooms with a class code, solves assignments in the editor, submits code with an optional comment, views marks and feedback	Immediate access after registration
Teacher	Creates classrooms and assignments, defines test cases, reviews submissions and awards marks	Access only after Admin approval (`is_approved = true`)
Admin	Pre-configured superuser who approves Teacher accounts and oversees the system	Created via setup / management command

2.3 Operating Environment

- Client: modern browsers (Chrome, Edge, Firefox, Safari: latest two versions), responsive for tablet and desktop.
- Server: Linux containers running Python 3.11+, Django, DRF, Celery workers and Redis.
- Database: PostgreSQL 14+.
- Sandbox: Docker containers (preferred) or restricted subprocess runner.

2.4 Design and Implementation Constraints

- Frontend: React.js, @monaco-editor/react, Axios, Chart.js / Recharts, Tailwind CSS.
- Backend: Python, Django REST Framework, SimpleJWT, custom DRF permissions (`IsAdmin`, `IsApprovedTeacher`, `IsClassroomStudent`), Celery + Redis.
- API documentation must be auto-generated with drf-spectacular (Swagger / OpenAPI).
- All student code must execute only inside the sandbox.

2.5 Assumptions and Dependencies

- An Admin account is pre-configured before first use.
- Docker (or a restricted runtime) is available on the Celery worker host.
- Redis and PostgreSQL services are available and reachable.
- Initial supported languages: Python (extensible to others via runner images).

---

3. System Features (Functional Requirements)

3.1 Registration, Authentication and Teacher Approval

Description: Users register with username, email, password and a role (Student or Teacher). Authentication uses JWT access/refresh tokens.

ID	Requirement
FR-1.1	The system shall allow registration with username, email, password and role (STUDENT or TEACHER).
FR-1.2	The system shall enforce unique username and email, and password strength validation.
FR-1.3	Student accounts shall be active immediately after registration.
FR-1.4	Teacher accounts shall be created with `is_approved = false` and shown the "Pending Approval" state.
FR-1.5	A Teacher with `is_approved = false` shall not access the Teacher Dashboard or create classrooms (HTTP 403).
FR-1.6	The Admin shall be able to list pending teachers and approve a teacher, after which full Teacher access is granted.
FR-1.7	The system shall issue JWT access and refresh tokens on login and support token refresh.

3.2 Classroom Management (Class Code System)

ID	Requirement
FR-2.1	An approved Teacher shall create a classroom/workspace with a name (e.g., "Python Lab 2026").
FR-2.2	The backend shall generate a unique class code for every classroom.

ID	Requirement
FR-2.3	A Student shall join a classroom by entering a valid class code, creating a classroom membership.
FR-2.4	The system shall reject invalid class codes and duplicate joins with clear error messages.
FR-2.5	Students shall see only classrooms they have joined; teachers only their own classrooms.

3.3 Assignment Management

ID	Requirement
FR-3.1	A Teacher shall create assignments scoped to one of their classrooms, with title, description, max marks and time limit.
FR-3.2	A Teacher shall add public and hidden test cases (input, expected output) to an assignment.
FR-3.3	Enrolled Students shall view assignments of their classroom; hidden test case data shall never be returned to students.

3.4 Code Editor and Submission

ID	Requirement
FR-4.1	Students shall write code in an integrated Monaco Editor in the browser.
FR-4.2	Students shall submit code with an optional Student Note / comment box.
FR-4.3	Submission shall be accepted only from students enrolled in the assignment's classroom.
FR-4.4	On submission the system shall store the code, set status to `SUBMITTED` and enqueue a Celery grading task, returning immediately.
FR-4.5	Students shall view submission status, test summary (public cases), teacher feedback and final marks once accepted.

3.5 Automated Grading Engine

ID	Requirement
FR-5.1	A Celery worker shall run the code in the sandbox against all public and hidden test cases.
FR-5.2	The sandbox shall enforce time limit, memory limit and restrict network, file-system and dangerous system calls.
FR-5.3	The engine shall compare actual and expected output per test case and record pass/fail, runtime and errors as JSON logs.
FR-5.4	The engine shall compute `automated_score` from the proportion of passed test cases and max marks.

ID	Requirement
FR-5.5	Timeouts, runtime errors and compilation errors shall be captured and reported without crashing the worker.
FR-5.6	After grading, the status shall change to `PENDING_REVIEW`.

3.6 Teacher Review and Approval Pipeline

ID	Requirement
FR-6.1	The Teacher Dashboard shall list submissions with status `PENDING_REVIEW` for the teacher's classrooms.
FR-6.2	The Teacher shall view the code, execution log, pass/fail metrics and student comment.
FR-6.3	The Teacher shall explicitly click "Accept / Approve Submission", enter final marks and optional feedback.
FR-6.4	On approval the status shall change to `ACCEPTED` (Accepted / Graded) and marks become visible to the student.
FR-6.5	The Teacher may reject a submission with feedback (`REJECTED`), allowing resubmission where permitted.
FR-6.6	Final marks shall not exceed the assignment's max marks.

3.7 Analytics

ID	Requirement
FR-7.1	A Teacher shall view classroom summary analytics: submission counts, average score, pass rate and status distribution.
FR-7.2	Analytics shall be rendered as charts (Chart.js / Recharts) on the dashboard.

---

4. Use Cases and Workflows

UC-1: Register and Obtain Teacher Approval

- **Actors:** Teacher, Admin
- **Precondition:** Admin account exists.
- **Main flow:**
    a. Teacher registers with role TEACHER.
    b. System creates the account with `is_approved = false` and shows "Pending Approval".
    c. Admin opens the pending teacher list and approves the teacher.
    d. System sets `is_approved = true`; the Teacher can now open the Teacher Dashboard and create classrooms.
- **Alternate flow:** Before approval, any teacher-only action returns 403 Forbidden.

UC-2: Create Classroom and Join with Class Code

- **Actors:** Teacher, Student
- **Main flow:**
    a. Approved Teacher creates classroom "Python Lab 2026".
    b. Backend generates a unique class code and displays it.
    c. Teacher shares the code with students.
    d. Student enters the code on the Join screen.
    e. System validates the code and creates a classroom membership.
- **Exception:** Invalid code → error "Invalid class code"; already joined → error "Already enrolled".

UC-3: Create Assignment

- **Actor:** Approved Teacher
- **Main flow:** Teacher selects a classroom → enters title, description, max marks, time limit → adds public and hidden test cases → saves. Enrolled students can now see the assignment.

UC-4: Solve and Submit Assignment

- **Actor:** Student
- **Main flow:**
    a. Student opens an assignment in a joined classroom.
    b. Writes code in the Monaco Editor.
    c. Optionally adds a Student Note explaining the approach.
    d. Clicks Submit; system stores the submission (`SUBMITTED`) and queues grading.
    e. Student sees "Submitted – awaiting grading".

UC-5: Automated Grading

- **Actor:** System (Celery worker)
- **Main flow:**
    a. Worker picks the task from the Redis queue.
    b. Runs the code in the sandbox for each test case with time and memory limits.
    c. Compares outputs, computes `automated_score`, stores JSON logs.
    d. Sets status to `PENDING_REVIEW`.
- **Exception:** Timeout or crash → test case marked failed with the error recorded; grading continues.

UC-6: Teacher Review and Approval

- **Actor:** Approved Teacher
- **Main flow:**
    a. Teacher opens the pending review queue.
    b. Opens a submission and reviews the code, logs, test metrics and student comment.
    c. Clicks "Accept / Approve Submission" and sets final marks (and feedback).
    d. System updates status to `ACCEPTED` and stores `final_marks`.
    e. Student sees "Accepted / Graded" with marks and feedback.
- **Alternate flow:** Teacher rejects with feedback → status `REJECTED`.

Submission Status Lifecycle

From	Event	To
(none)	Student submits	SUBMITTED
SUBMITTED	Sandbox grading completes	PENDING_REVIEW
PENDING_REVIEW	Teacher approves and awards marks	ACCEPTED
PENDING_REVIEW	Teacher rejects with feedback	REJECTED

---

5. System Architecture

Layer	Technology	Responsibility
Presentation	React.js, Tailwind CSS, @monaco-editor/react, Axios, Chart.js / Recharts	UI, editor, dashboards, API calls
API	Django REST Framework, SimpleJWT, custom permissions	REST endpoints, authentication, RBAC, validation
Task Queue	Celery + Redis	Asynchronous grading jobs
Execution Sandbox	Docker / Subprocess isolation	Safe execution with time, memory and syscall restrictions
Data	PostgreSQL	Users, classrooms, memberships, assignments, test cases, submissions, JSON logs
Documentation	drf-spectacular	Swagger / OpenAPI schema

Request flow: React → DRF API (JWT + permission check) → save submission → Celery task via Redis → sandbox execution → results to PostgreSQL → Teacher review → final marks visible to student.

---

6. Database Schema

6.1 Entities

User (extends `AbstractUser`)

Field	Type	Constraints / Notes
id	BigAutoField	PK
username	CharField	Unique
email	EmailField	Unique
password	CharField	Hashed

Field	Type	Constraints / Notes
role	CharField	Choices: STUDENT, TEACHER
is_approved	BooleanField	Used for teachers; default false for TEACHER, true for STUDENT

Classroom

Field	Type	Constraints / Notes
id	BigAutoField	PK
name	CharField	Required
class_code	CharField	Unique, auto-generated
teacher	FK → User	Approved teacher
created_at	DateTimeField	Auto

ClassroomStudent (junction)

Field	Type	Constraints / Notes
id	BigAutoField	PK
classroom	FK → Classroom	
student	FK → User	
joined_at	DateTimeField	Auto
(classroom, student)		Unique together

Assignment

Field	Type	Constraints / Notes
id	BigAutoField	PK
classroom	FK → Classroom	
title	CharField	Required
description	TextField	Problem statement
max_marks	PositiveIntegerField	
time_limit	PositiveIntegerField	Seconds per execution

TestCase

Field	Type	Constraints / Notes
id	BigAutoField	PK
assignment	FK → Assignment	

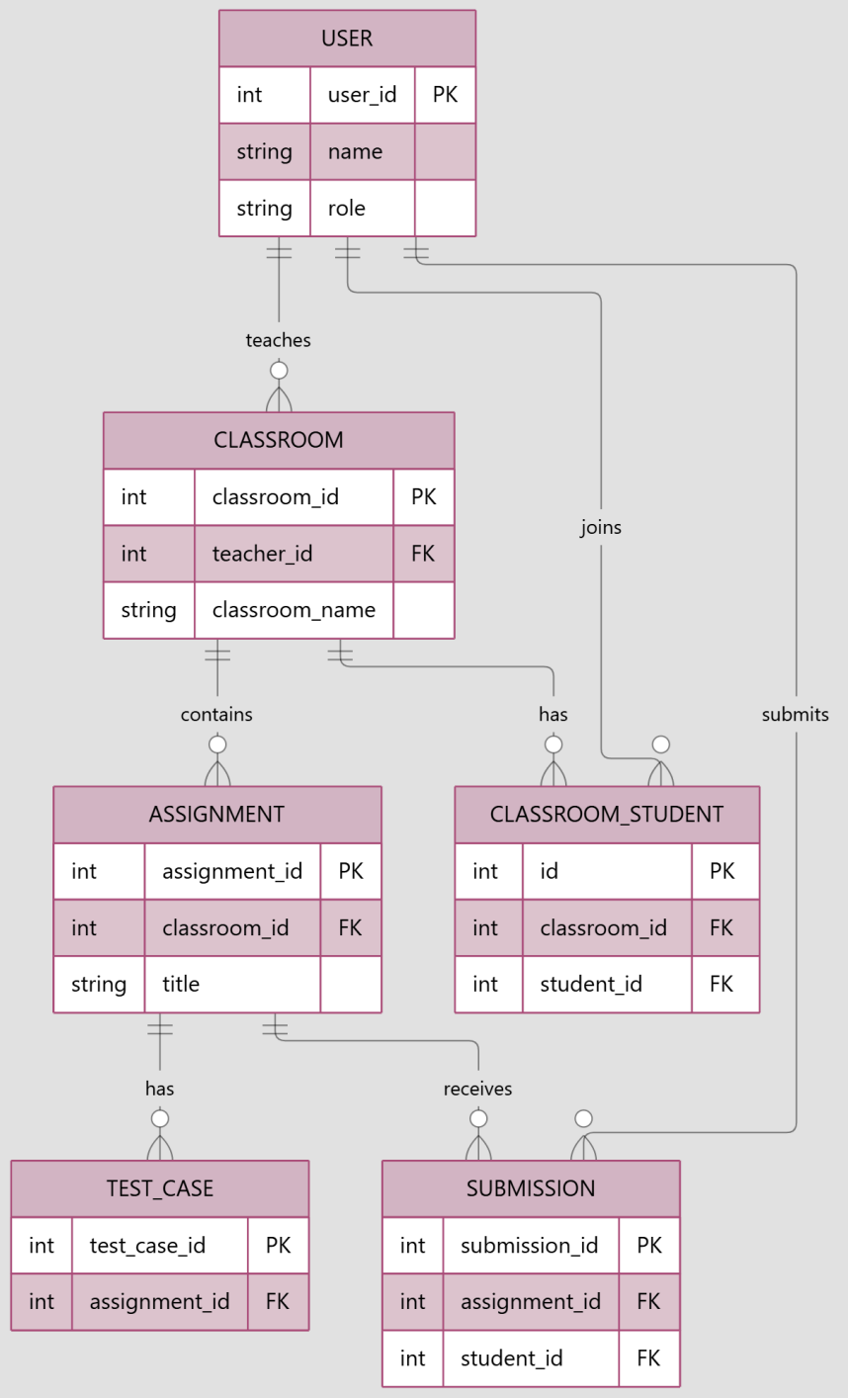
Field	Type	Constraints / Notes
input_data	TextField	
expected_output	TextField	
is_hidden	BooleanField	Hidden cases never exposed to students

Submission

Field	Type	Constraints / Notes
id	BigAutoField	PK
assignment	FK → Assignment	
student	FK → User	
code	TextField	Submitted source
student_comment	TextField	Optional note
automated_score	DecimalField	Set by grader
execution_log	JSONField	Per-test results, runtime, errors
status	CharField	SUBMITTED, PENDING_REVIEW, ACCEPTED, REJECTED
final_marks	DecimalField	Null until approved
teacher_feedback	TextField	Optional
submitted_at	DateTimeField	Auto

6.2 Relationships

- User (Teacher) 1 — N Classroom
- Classroom N — N User (Student), via ClassroomStudent
- Classroom 1 — N Assignment
- Assignment 1 — N TestCase
- Assignment 1 — N Submission; User (Student) 1 — N Submission



7. API Endpoints Specification

Base path: `/api/v1/`. Authentication: JWT Bearer token unless stated otherwise.

#	Method	Endpoint	Permission	Request	Response
1	POST	`/auth/register/`	Public	`{username, email, password, role}`	201 `{id, username, role, is_approved}`
2	POST	`/auth/login/`	Public	`{username, password}`	200 `{access, refresh}`
3	PATCH	`/admin/approve-teacher/{id}/`	IsAdmin	none (or `{is_approved: true}`)	200 `{id, is_approved: true}`
4	POST	`/classrooms/`	IsApprovedTeacher	`{name}`	201 `{id, name, class_code}`
5	POST	`/classrooms/join/`	Student	`{class_code}`	200 `{classroom_id, name}`; 400 if invalid
6	POST	`/classrooms/{id}/assignments/`	IsApprovedTeacher (owner)	`{title, description, max_marks, time_limit, test_cases[]}`	201 assignment object
7	GET	`/classrooms/{id}/assignments/`	Enrolled Student / owner Teacher	none	200 list (hidden test cases excluded for students)
8	POST	`/assignments/{id}/submit/`	IsClassroomStudent	`{code, student_comment?}`	202 `{submission_id, status: SUBMITTED}`
9	GET	`/teacher/submissions/pending/`	IsApprovedTeacher	none	200 list `{id, student, assignment, automated_score, student_comment, status}`
10	PATCH	`/teacher/submissions/{id}/review/`	IsApprovedTeacher (owner)	`{action: ACCEPT or REJECT, final_marks, teacher_feedback?}`	200 `{id, status, final_marks}`
11	GET	`/analytics/classroom-summary/?classroom_id=X`	IsApprovedTeacher (owner)	query param	200 `{total_submissions, avg_score, pass_rate, status_counts}`

8. External Interface Requirements

8.1 User Interfaces

- Responsive web UI built with Tailwind CSS.
- Role-based dashboards: Student (classrooms, assignments, results), Teacher (classrooms, assignments, review queue, analytics), Admin (pending teacher approvals).
- Assignment page with problem statement, Monaco Editor, comment box and Submit button.
- Teacher review page showing code, test results, student comment and Approve/Reject controls.

8.2 Software Interfaces

Interface	Direction	Purpose	Protocol
Frontend ↔ Backend	Bidirectional	All application data	REST/JSON over HTTPS
Backend ↔ Redis	Bidirectional	Task queue messaging	Redis protocol
Backend ↔ PostgreSQL	Bidirectional	Persistence	PostgreSQL protocol
Worker ↔ Docker	Outbound	Sandbox execution	Docker Engine API

8.3 Communications Interfaces

All client-server communications are using HTTPS.

---

9. Non-Functional Requirements

9.1 Performance

- Standard API responses within 2 seconds under normal load.
- Submission API shall return within 1 second since grading is asynchronous.
- A typical submission shall be graded within 30 seconds of queuing.

9.2 Security

- Passwords hashed (PBKDF2/Argon2); JWT with short-lived access tokens.
- RBAC is enforced on every endpoint through custom DRF permissions.
- Hidden test cases and other students' code shall never be exposed to unauthorized users.
- Sandbox: no network access, read-only file system except a temporary directory, CPU/memory/time limits, blocked dangerous system calls, non-root execution, container destroyed after each run.
- Protection against SQL injection, XSS and CSRF; CORS restricted to the frontend origin.

9.3 Reliability and Availability

- Failed or crashed grading tasks shall be retried or marked with an error without blocking the queue.
- Target availability of 99% on the deployed environment.

9.4 Scalability

- Celery workers shall scale horizontally to handle concurrent submissions.

9.5 Usability

- A new student shall be able to join a class and submit a first solution within 5 minutes without training.

9.6 Maintainability and Compatibility

- Modular Django apps (accounts, classrooms, assignments, submissions, grader, analytics), PEP 8 compliance, and compatibility with the latest two versions of major browsers.

---

10. Testing Strategy

Framework: Django `TestCase` and PyTest (with pytest-django).

Level	Scope	Example Scenarios
Unit	Business logic, models, serializers, sandbox runner	Class code uniqueness; score calculation; output comparison; sandbox returns timeout error
Integration	API flows and permissions	Class code validation and join; RBAC for each role; submit → grade → pending review → teacher approval → marks visible
Edge-case	Failure and abuse cases	Unapproved teacher creating classroom (403); invalid class code; duplicate join; infinite-loop code (timeout); memory-heavy code; non-enrolled student submitting; teacher reviewing another teacher's submission; marks above max
Security	Sandbox escape attempts	Network calls, file access, forbidden system calls
Frontend	UI behaviour	Editor loads, submission feedback, dashboard rendering

---

11. Deployment Plan

Component	Platform
Frontend (React)	Vercel
Backend API (Django + DRF)	Render
Celery Worker	Render
Redis	Managed Redis add-on
Database	Managed PostgreSQL
API Docs	Swagger UI at `/api/docs/`

- Configuration through environment variables (secret key, DB URL, Redis URL, CORS origins).
- CI pipeline runs the test suite before each deployment.
- Database migrations run on each release; the Admin account is created via a management command.m

12. Appendix A: Requirements Traceability

Requirement Group	Feature	Use Case	API Endpoints
FR-1	Registration and teacher approval	UC-1	1, 2, 3
FR-2	Classroom and class code	UC-2	4, 5
FR-3	Assignment management	UC-3	6, 7
FR-4	Editor and submission	UC-4	8
FR-5	Automated grading	UC-5	(internal Celery task)
FR-6	Teacher review and approval	UC-6	9, 10
FR-7	Analytics	UC-6 (supporting)	11